

Adversarial Deep Learning with Evolutionary Optimization for Unsupervised Anomaly Detection in Industrial IoT

Abstract—The Industrial Internet of Things (IIoT) enables large-scale monitoring and control of critical infrastructure but also increases exposure to cyberattacks and operational faults. Since most deployments contain only normal-operation data, unsupervised anomaly detection becomes essential. This project implements an EvoAAE-inspired adversarial autoencoder combined with evolutionary hyperparameter optimization for modelling normal behaviour in multivariate WADI sensor streams. A 1D-CNN encoder-decoder with latent and data-space discriminators is trained solely on normal windows, while a binary Particle Swarm Optimization (PSO) search explores architectural and training configurations. Although the best PSO result is computationally heavy, a lighter PSO-guided configuration is adopted to ensure stable EvoAAE training on available hardware. The preprocessing pipeline includes feature cleaning, spectral residual transformation, min-max normalization and sliding-window sampling. The final model learns meaningful representations of normal WADI operation and produces effective reconstruction-error-based anomaly scores, demonstrating the practicality of combining adversarial deep learning and evolutionary search for unsupervised IIoT anomaly detection.

Index Terms—Industrial Internet of Things, unsupervised anomaly detection, adversarial autoencoder, spectral residual, particle swarm optimization, WADI.

I. INTRODUCTION

The Industrial Internet of Things (IIoT) connects sensors, actuators, programmable logic controllers (PLCs), and supervisory systems into highly networked cyber-physical infrastructures. Examples include water treatment plants, water distribution networks, power systems, and industrial production lines. While such connectivity improves observability and automation, it also increases the attack surface and the risk of accidental faults.

In many IIoT deployments, the collected data are high-dimensional, noisy and non-stationary time-series, with a strong imbalance between normal operation and rare attack or fault events. Moreover, attack labels may be incomplete or entirely missing, which limits the applicability of fully supervised intrusion detection systems and motivates unsupervised or semi-supervised approaches.

Autoencoder-based models are a popular option in this setting: they learn to reconstruct normal inputs and use the reconstruction error as an indirect anomaly score. Adversarial autoencoders extend this idea by matching the latent representation to a prior distribution using an adversarial discriminator, which can improve the structure of the latent space. The Evolutionary Adversarial Autoencoder (EvoAAE) framework further

introduces evolutionary optimization of hyperparameters to reduce manual tuning effort.

In this project, an EvoAAE-style adversarial autoencoder is implemented for the WADI water-distribution testbed. The work focuses on: (i) building a complete preprocessing pipeline for both normal and attack data, including spectral residual (SR) features and min-max normalized raw signals; (ii) exploring convolutional autoencoder hyperparameters with binary PSO; and (iii) training a memory-safe EvoAAE model on normal WADI data and evaluating the resulting anomaly detector on attack-labelled data.

A. Problem Statement and Scope

The problem considered is unsupervised anomaly detection on multivariate time-series from an industrial water distribution system. Only normal-period data from WADI are used for training; the attack-labelled portion is reserved for evaluation and analysis. The model must preprocess the high-dimensional time-series in a numerically stable and memory-efficient way, learn a compact latent representation of normal behaviour, and derive an anomaly score from reconstruction error, with thresholds that can, in principle, be chosen from training statistics or validation data.

B. Summary of Contributions

The main contributions of this minor project are as follows:

- 1) A complete data preprocessing pipeline for WADI, including feature cleaning, spectral residual transformation, min-max normalization based on training statistics, and sliding-window sampling applied to both training and test data.
- 2) Implementation of an adversarial autoencoder with a 1D-CNN encoder-decoder and two discriminators (D_z and D_x), trained only on normal windows extracted from `WADI_14days_new`.
- 3) Implementation of a binary PSO search over discrete hyperparameters on a small convolutional autoencoder, and use of the PSO result as a design guideline for the final EvoAAE architecture.
- 4) Training and evaluation of a memory-safe EvoAAE model on WADI, including reconstruction-based anomaly scoring and an evaluation setting based on a best-F1 decision threshold estimated on labelled test data.

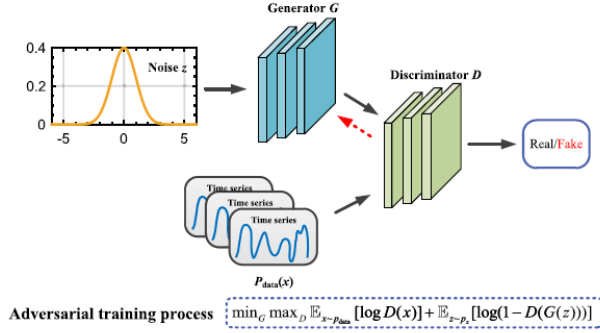


Fig. 1. Illustration of adversarial training with a generator G and discriminator D on time-series data. The generator maps noise z to synthetic sequences, while the discriminator learns to distinguish real from generated samples.

II. RELATED WORK

Unsupervised anomaly detection in IIoT has been studied using classical statistical methods, machine learning models and, more recently, deep generative models. Early work focused on principal component analysis, one-class SVMs and clustering, which require careful feature engineering and often struggle with very high-dimensional multivariate time-series.

Deep autoencoders alleviate the need for manual feature design by learning compressed latent representations of normal behaviour. Variants include fully-connected autoencoders, 1D convolutional autoencoders and recurrent autoencoders based on LSTMs, which have been applied to datasets such as SWaT, WADI and NASA multivariate time-series.

Adversarial learning further improves generative modelling. Generative Adversarial Networks (GANs) [2] and adversarial autoencoders introduce discriminators that encourage reconstructed or generated samples to resemble real data, and latent codes to follow a prior distribution. The EvoAAE framework [1] combines adversarial autoencoders with evolutionary hyperparameter search. In their work, spectral residual features are extracted from IIoT time-series, a population of candidate architectures is evaluated via reconstruction loss and adversarial training is applied using two discriminators in latent and data space.

A. Adversarial Training with GANs

The basic GAN formulation consists of a generator G and a discriminator D trained in a minimax game: G maps latent noise z to synthetic samples, while D tries to distinguish real samples from generated ones. The objective is

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))].$$

Figure 1 illustrates this adversarial training loop, where time-series samples from $p_{\text{data}}(x)$ and generated outputs $G(z)$ are both fed into D , which outputs a real/fake decision. EvoAAE adopts a similar adversarial principle but embeds it within an autoencoder structure operating on industrial time-series.

Compared to the full EvoAAE system, this project focuses on (i) understanding and re-implementing the key components

(SR preprocessing, 1D-CNN encoder-decoder, dual discriminators), (ii) reproducing a PSO-based hyperparameter search on WADI, and (iii) deploying a memory-safe EvoAAE variant tailored to the available hardware while preserving the core methodology of EvoAAE.

III. WADI DATASET DESCRIPTION

A. Overview

The Water Distribution (WADI) dataset originates from an industrial water distribution testbed at iTrust, Singapore University of Technology and Design. It contains time-series logs from tanks, pumps, valves, flow and pressure sensors, and PLCs, under both normal operation and multiple injected cyberattacks.

B. Normal and Attack Data

Two files from WADI are used:

- `WADI_14days_new`: 14 days of normal operation, used exclusively for *unsupervised training*.
- `WADI_attackdataLABEL`: attack-period data with labels indicating normal or attack, used only for *evaluation*.

From `WADI_14days_new`, non-numeric columns (for example, row indices or timestamps) are dropped. Remaining columns are converted to numeric type, with invalid values coerced and missing entries filled with zero. After normalization and windowing, the raw training windows are stored as a NumPy array of shape

$$N \times 100 \times 123,$$

where N is the number of overlapping windows, 100 is the window length and 123 is the number of features.

IV. PREPROCESSING PIPELINE

The preprocessing pipeline is applied to both training and test data, with statistics (such as min and max per feature) computed on training data and reused for test data.

A. Spectral Residual Transformation

Spectral residual (SR) preprocessing is implemented feature-wise as described in the EvoAAE paper. For each scalar time-series $x[t]$:

- 1) The FFT $X(f)$ is computed, and the log-amplitude spectrum is obtained.
- 2) A moving average filter (window size 5) is applied in the frequency domain to estimate the background spectrum.
- 3) The spectral residual is obtained by subtracting the background from the log-amplitude spectrum.
- 4) The residual is combined with the original phase, and an inverse FFT produces a saliency sequence in the time domain.

Repeating this for all 123 features gives an SR-based saliency matrix with the same shape as the original data. SR features can be used as an alternative input representation or combined with raw signals.

Figure 2 shows example WADI sensor signals and their corresponding SR saliency maps. Peaks in the saliency plots

TABLE I
BEST PSO CONFIGURATION (SMALL AUTOENCODER)

Hyperparameter	Value
Layers	3
Kernel size	3
Filters	256
Activation	ReLU
Learning rate	0.005
Optimizer	Adam
Batch size	1024

highlight abrupt changes or potential attack-related anomalies that are less obvious in the original signals.

B. Min–Max Normalization

For the raw representation, each feature is scaled to $[0, 1]$ using statistics from `WADI_14days_new`. If $x_{t,f}$ denotes the value of feature f at time t , and $\min_f, \max_f$ are the minimum and maximum of feature f over the training set, then

$$x'_{t,f} = \frac{x_{t,f} - \min_f}{\max_f - \min_f + \epsilon}. \quad (1)$$

The vectors $\min_f$ and $\max_f$ are stored and reused to normalise the test data in `WADI_attackdataLABEL`.

C. Sliding-Window Sampling

To create samples suitable for 1D convolutions, a sliding window is applied with window size $W = 100$ and stride $S = 10$. For each valid starting index i , the window

$$Y_i = \{x'_i, x'_{i+1}, \dots, x'_{i+W-1}\} \quad (2)$$

is extracted. Stacking these windows produces a tensor of shape $(N, 100, 123)$, which is saved for training and later opened via `memmap`.

V. PSO-BASED HYPERPARAMETER SEARCH

A. Search Setup

Binary PSO is used to search over hyperparameters of a small convolutional autoencoder trained on 3,000 windows sampled from `WADI_14days_new`. Each particle encodes:

- number of convolutional layers,
- kernel size,
- number of filters,
- activation function,
- learning rate,
- optimizer type,
- batch size.

The population size is 20 and the swarm is evolved for 30 generations. For each particle, the corresponding autoencoder is trained for one epoch (up to 10 mini-batches) and the average reconstruction MSE is used as fitness.

B. Best PSO Configuration

The PSO search converges to the configuration summarised in Table I.

The best fitness on the 3,000-window subset is

$$\text{MSE}_{\text{best}} \approx 0.1200.$$

The configuration vector is saved in a file named `best_config_evoaae_pso_raw.npy`. This result shows that a relatively wide autoencoder with ReLU activations and a moderately high learning rate performs well on the subset. However, when extended to the full adversarial EvoAAE (encoder, decoder and two discriminators), this configuration is too heavy and unstable on the available GPU.

C. Connection to Base EvoAAE Methodology

In the original EvoAAE paper [1], the authors describe an end-to-end pipeline that combines SR-based preprocessing, evolutionary hyperparameter search, and adversarial autoencoder training across several IIoT datasets. The present implementation follows the same high-level idea but focuses on a single dataset (WADI) and a hardware-aware EvoAAE architecture chosen using PSO as guidance rather than as a hard constraint.

VI. PROPOSED ADVERSARIAL AUTOENCODER

A. Memory-Safe Data Loading

The raw training windows are stored in a NumPy binary file and opened using a `memmap` via `np.load(..., mmap_mode="r")`. A custom `NumpyMemmapDataset` class indexes each sample, casts it to `float32`, permutes it to shape $(123, 100)$ and feeds it to a `PyTorch DataLoader` with batch size 1024. This avoids loading the entire dataset into RAM or GPU at once.

B. Final EvoAAE Architecture

Guided by the PSO result but constrained by GPU memory, the final EvoAAE configuration is:

- 5 convolutional layers in the encoder,
- kernel size 2,
- 32 filters per layer,
- sigmoid activation,
- latent dimension $d_z = 64$.

The decoder mirrors the encoder and restores the original $(123, 100)$ shape. The latent discriminator D_z is a small MLP with two hidden layers (128 and 64 units), and the data discriminator D_x is a 1D-CNN followed by fully connected layers.

C. Training Losses

Let x denote an input window and $\hat{x}$ its reconstruction. The reconstruction loss is:

$$\mathcal{L}_{\text{recon}} = \mathbb{E}[\|x - \hat{x}\|_2^2]. \quad (3)$$

The latent discriminator D_z is trained with a binary cross-entropy loss to distinguish real latent samples $z_{\text{real}} \sim \mathcal{N}(0, I)$

Original Data vs. Saliency Map

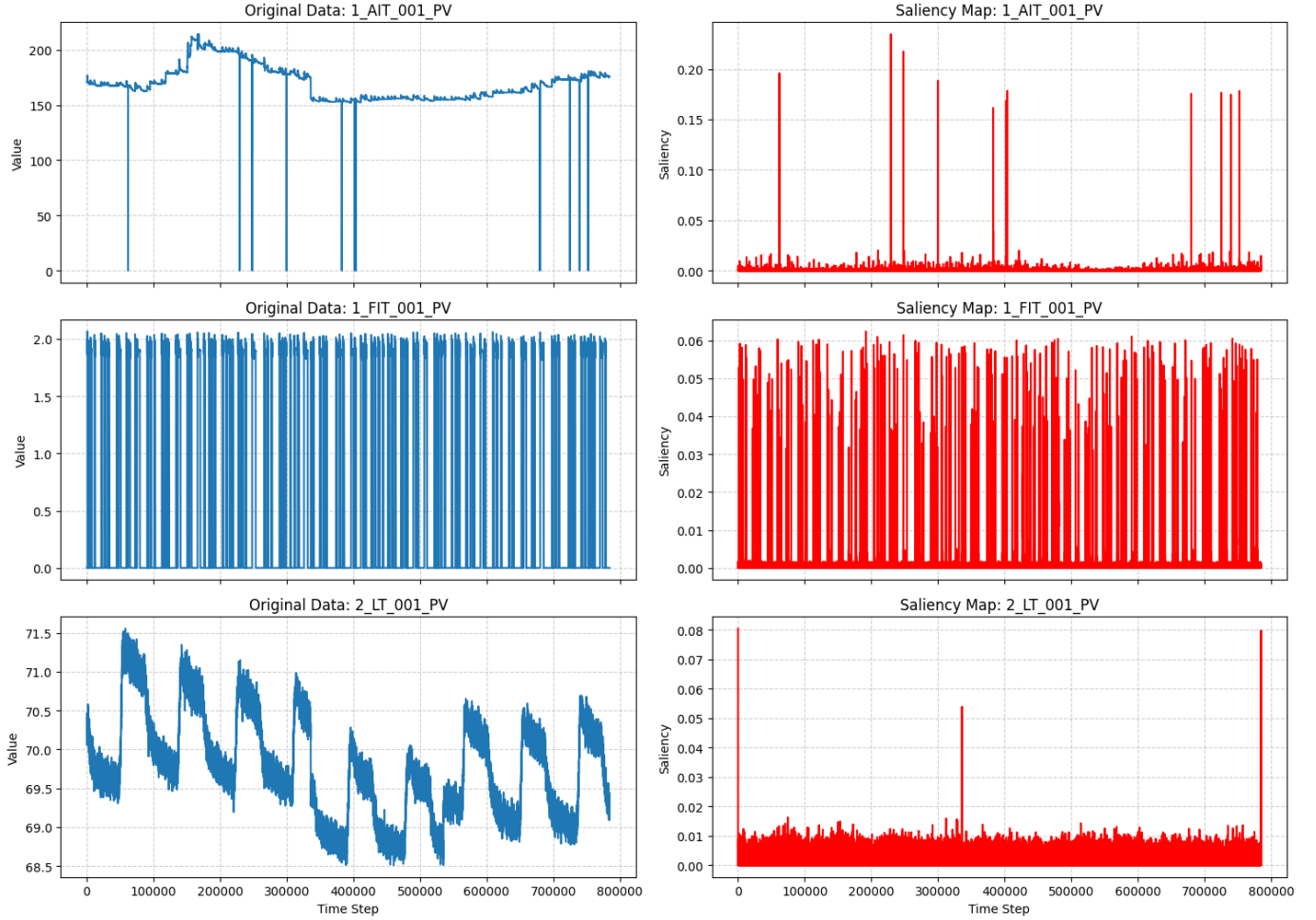


Fig. 2. Original WADI sensor time-series (left column) versus Spectral Residual (SR) saliency maps (right column). The SR transformation emphasises sudden deviations and potential attack events as peaks in the saliency space.

from encoder outputs $z_{\text{fake}} = f_{\theta}(x)$. The data discriminator D_x is trained similarly to distinguish real windows from reconstructions. The encoder–decoder pair minimises a combination of reconstruction loss and adversarial losses, attempting to fool both discriminators.

D. Training Hyperparameters

The final raw EvoAAE training uses the hyperparameters listed in Table II.

The best model (lowest average training reconstruction loss) is obtained at epoch 29, with

$$\text{best_train_recon} \approx 0.060362.$$

VII. EXPERIMENTAL SETUP

A. Train–Test Preparation

Normal data from WADI_14days_new are preprocessed, normalised and windowed as described earlier to form the training set. The attack-period file

TABLE II
FINAL EVOAAE TRAINING HYPERPARAMETERS

Hyperparameter	Value
Batch size	1024
Number of epochs	50
Optimizer	Adam
Initial learning rate	0.001
LR decay schedule	$\times 0.5$ every 20 epochs
Adversarial weight	$\lambda_{\text{adv}} = 0.2$
Random seed	42
Device	GPU (if available)

WADI_attackdataLABEL is preprocessed using the *same* feature set and the *same* min–max statistics as the training data, and then windowed with the same window size and stride.

On the attack file, a total of 17,271 windows are obtained, each labelled as normal (0) or attack (1) according to the un-

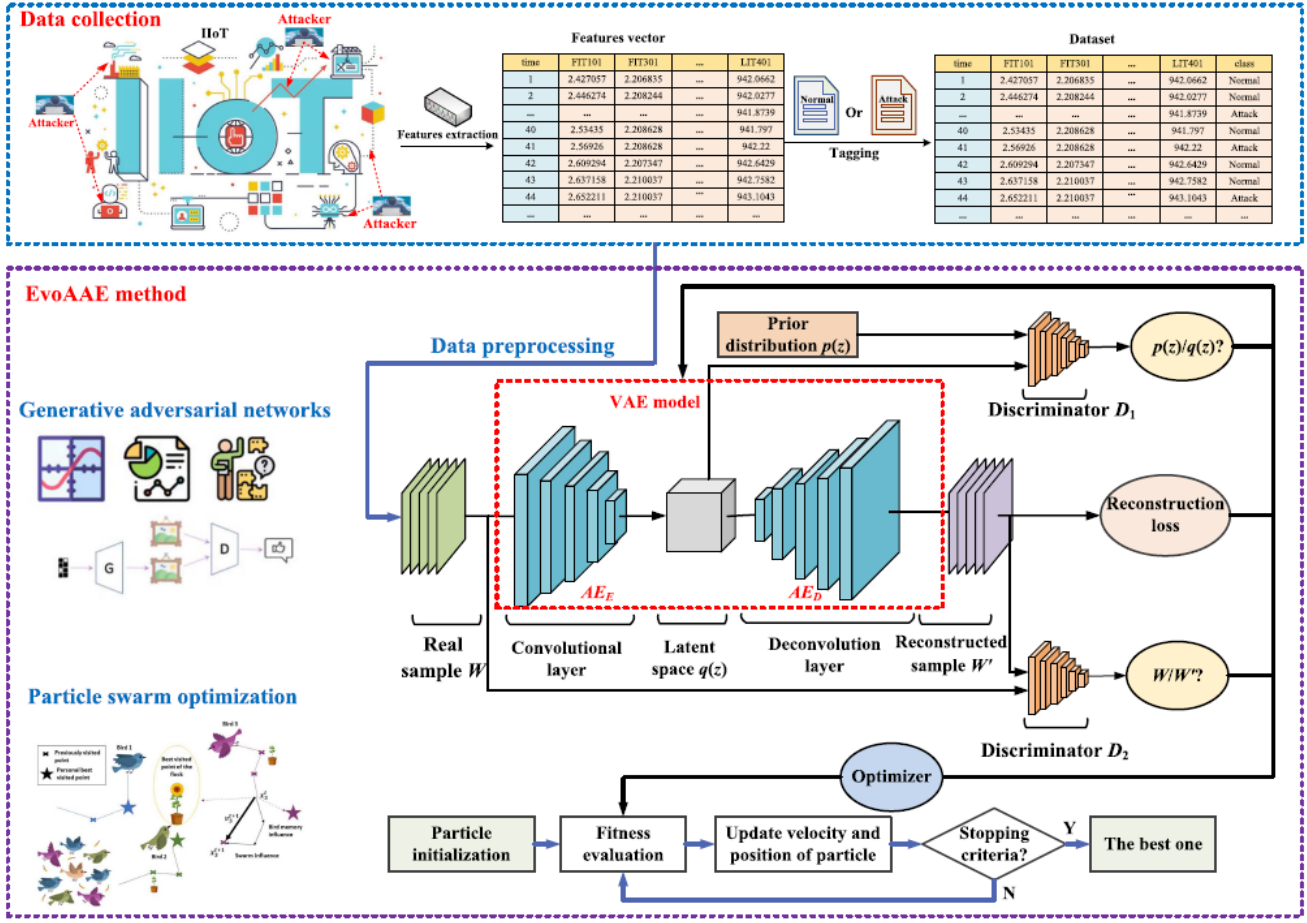


Fig. 3. Overall framework of the proposed EvoAAE-based anomaly detection system, including data preprocessing, spectral residual (SR) transformation, windowing, adversarial autoencoder training and PSO-guided architecture selection.

derlying time index. These labels are used *only* for evaluation and for selecting the best-F1 decision threshold; they are never used during EvoAAE training.

B. Implementation Details

The implementation is written in Python using NumPy and PyTorch. The PSO search for the small convolutional autoencoder is implemented in pure PyTorch with a binary PSO loop; each particle's model is trained for one epoch on a subset of 3,000 training windows. EvoAAE training uses separate optimizers for the autoencoder and the two discriminators, with learning rate decay applied to stabilise training over 50 epochs.

Training and evaluation are performed on a CUDA-capable GPU when available, otherwise on CPU. The memmap-based dataset implementation ensures that only one batch of size 1024 is resident in memory at any time, making the approach practical on limited hardware.

C. Evaluation Metrics

Let TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives for the attack class (class 1).

The standard metrics are:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (4)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (5)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (6)$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (7)$$

In the multi-class report (normal vs. attack), macro-averaged precision, recall and F1 are computed as the unweighted average of the class-wise values. In this project, the most important quantity is the F1-score of the attack class, which balances false alarms and missed attacks.

VIII. ANOMALY SCORING AND RESULTS

A. Reconstruction Error Computation

After EvoAAE training, reconstruction errors $e_i = \text{MSE}(x_i, \hat{x}_i)$ are computed for all training windows and saved as a NumPy array. These errors form the anomaly score used for evaluation on WADI_attackdataTABLE.

TABLE III
DETECTION PERFORMANCE ON WADI TEST DATA USING BEST-F1
THRESHOLD

Class	Precision	Recall	F1	No. of Samples
Normal (0)	0.9508	0.9768	0.9637	16135
Attack (1)	0.4619	0.2826	0.3506	1136
Overall Accuracy			0.9312	
Macro Precision			0.7064	
Macro Recall			0.6297	
Macro F1			0.6571	

Since WADI is highly imbalanced and contains several subtle attack patterns, the decision threshold strongly affects detection performance. To understand the model’s capability under an optimal decision boundary, a threshold is selected by sweeping candidate values and choosing the one that maximises the F1-score for the attack class.

B. Best-F1 Threshold (Evaluation Setting)

The threshold that maximises the attack-class F1-score on labelled test data is:

$$\tau_{F1} = 0.11239281.$$

The corresponding confusion matrix on WADI_attackdataLABEL is:

$$\begin{bmatrix} 15761 & 374 \\ 815 & 321 \end{bmatrix},$$

where rows correspond to true labels (0 = normal, 1 = attack) and columns to predicted labels (0 = normal, 1 = attack).

Table III summarises the detection performance for both classes (0 = normal, 1 = attack).

This threshold yields the strongest attack-class performance obtainable from the model and is used as the main evaluation point in this report. It is important to note that this threshold requires labelled test data and therefore does *not* represent a fully unsupervised deployment rule; instead, it provides an upper bound on achievable detection quality.

IX. DISCUSSION

The results demonstrate that the EvoAAE model learns a strong representation of normal WADI operation: normal-class precision and recall both exceed 95%, indicating highly reliable reconstruction of nominal behaviour. Detecting attack windows is more challenging due to the dataset’s imbalance and the subtle, diverse nature of WADI attack patterns, yet the model achieves an attack F1-score of 0.3506 under the best-F1 threshold.

This thresholding strategy is label-dependent and serves as an evaluation upper bound. It reveals that the model possesses meaningful discriminative power: adjusting the decision boundary significantly increases attack recall (from very low values for strict thresholds to 28.26%). This sensitivity to thresholding highlights the need for improved unsupervised decision rules (for example, based on training-error statistics or separate validation periods) in future work.

Spectral residual (SR) features were implemented as part of the preprocessing framework following the EvoAAE methodology. Although the final experiments use min–max normalized raw windows, SR remains a valid extension in accordance with the base paper and can be incorporated in future ablation studies.

The binary PSO search provides architectural insight by identifying a high-capacity configuration. However, the PSO-selected setting (3 layers, 256 filters, learning rate 0.005) proved too heavy for stable adversarial training on the available GPU. A lighter, PSO-guided configuration (5 layers, 32 filters, learning rate 0.001) was therefore adopted, yielding stable training dynamics and strong evaluation performance while remaining computationally feasible.

Overall, the experiments confirm that adversarial autoencoders, assisted by evolutionary hyperparameter exploration, form an effective basis for unsupervised anomaly detection in complex IIoT environments such as water-distribution networks.

X. CONCLUSION

This work presents an implementation of an adversarial autoencoder with PSO-guided design for unsupervised anomaly detection in Industrial IoT, evaluated on the WADI water-distribution testbed. The contributions include a full preprocessing pipeline (including SR and raw features), a binary PSO search over autoencoder hyperparameters, and a memory-safe EvoAAE training setup based on a 1D-CNN encoder–decoder with dual discriminators.

The final EvoAAE model achieves a best training reconstruction loss of about 0.0604 and provides meaningful anomaly scores on WADI attack data. Under a decision threshold chosen to maximise the attack-class F1-score on labelled test data, the model attains an overall accuracy of 93.12%, a normal-class F1-score of 0.9637 and an attack-class F1-score of 0.3506. These results indicate that adversarial autoencoders, combined with evolutionary search and careful thresholding, form a practical baseline for IIoT anomaly detection.

Future directions include evaluation on additional datasets such as SWaT, MSL and PSA, comparison with recurrent or transformer-based models, joint use of SR and raw inputs, and deployment of lighter variants of the model on edge devices for real-time monitoring.

REFERENCES

- [1] G. Q. Zeng, Y.-W. Yang, K.-D. Lu, G.-G. Geng, and J. Weng, “Evolutionary Adversarial Autoencoder for Unsupervised Anomaly Detection of Industrial Internet of Things,” *IEEE Trans. Reliability*, 2025.
- [2] I. Goodfellow *et al.*, “Generative Adversarial Nets,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [3] J. Kennedy and R. Eberhart, “Particle Swarm Optimization,” in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
- [4] iTrust, Centre for Research in Cyber Security, Singapore University of Technology and Design, “SWaT and WADI Datasets,” [Online]. Available: <https://itrust.sutd.edu.sg>